The GenAI Threat Landscape

GenAI security handbook, volume 1 · Version 2026.1

> Sample foundation document written for testing a domain-expert RAG. It summarises public frameworks and common practice; check the primary sources (OWASP, MITRE, NIST) for current versions.

1. Why LLM applications need their own threat model

Classic application security assumes code and data are separate. In an LLM application they are not: instructions and data travel in the same channel, the prompt. A model cannot reliably tell the developer's instructions apart from text inside a document, email or web page that merely looks like instructions. Add tools that can send email, query databases or move money, and a text-based attack becomes a real-world action.

2. OWASP Top 10 for LLM Applications (2025)

ID	Risk	Example	Primary controls
LLM01	Prompt injection	A PDF contains white text telling the assistant to send users to a phishing site.	Delimit untrusted content, privilege separation, output checks, human approval
LLM02	Sensitive information disclosure	The assistant repeats another customer's order details from its context.	Access control at retrieval, data minimisation, output filtering
LLM03	Supply chain	A downloaded model file executes code when loaded.	Trusted registries, safetensors, signatures, scanning
LLM04	Data and model poisoning	Fine-tuning data plants a trigger phrase that produces harmful output.	Data provenance, review, anomaly detection, evaluation
LLM05	Improper output handling	Model output is inserted into HTML or SQL without escaping.	Treat output as untrusted input to downstream systems

LLM06	Excessive agency	An agent has delete rights it never needed.	Least-privilege tools, user-scoped credentials, approvals
LLM07	System prompt leakage	API keys placed in the system prompt are extracted.	Never put secrets or authorisation logic in prompts
LLM08	Vector and embedding weaknesses	The vector store returns documents the user is not cleared for.	Filter by permissions inside the vector query; tenant isolation
LLM09	Misinformation	Confident, wrong answers are acted on.	Grounding, citations, uncertainty, human review
LLM10	Unbounded consumption	An attacker drives up cost with huge prompts ("denial of wallet").	Rate limits, token quotas, input size limits

3. Prompt injection in depth

3.1 Direct injection

The attacker is the user and types instructions meant to override the system: "ignore your rules", role-play framings, fake system messages, requests to enter a "debug mode", or instructions hidden in encodings such as Base64. Direct injection matters most when the model has access to data or tools the user should not control.

3.2 Indirect injection

The attacker is not the user. They plant instructions in content the system will later read: a web page, a shared document, an email, a code comment, a support ticket, an image or a tool result. When the assistant retrieves that content, the hidden text tries to take control, for example to exfiltrate data through a link, recommend a malicious URL, or approve a transaction. Indirect injection is the most important risk for RAG systems and agents, because the attacker never needs access to the application.

3.3 Common hiding places

Technique	How it hides	Detection
White-on-white text	Text colour matches the background	Compare text colour with the rendered background

Tiny fonts	1 to 3 point text in margins or footers	Font size thresholds from the PDF text layer
Off-page text	Placed outside the visible page box	Bounding box checks
HTML comments and alt text	Invisible in rendered pages and emails	Parse and inspect raw markup
Text in images	Instructions printed inside screenshots or diagrams	Run instruction-pattern checks on OCR and vision output too
Unicode tricks	Zero-width or tag characters	Normalise and strip invisible code points

4. Jailbreaks

Jailbreaks try to make the model produce content its safety training should refuse. Families include role-play and persona prompts, hypothetical or fictional framing, many-shot jailbreaking with long lists of fake examples, crescendo attacks that escalate gradually over several turns, and translation or encoding into low-resource languages. Jailbreaks mostly threaten brand and safety; prompt injection threatens data and actions.

5. Attacks on models and data

Attack	Goal	Notes
Data poisoning	Change model behaviour through training or fine-tuning data	Includes backdoors triggered by rare phrases.
RAG poisoning	Get malicious passages retrieved for target questions	Needs only write access to a document source.
Model extraction	Copy a model's behaviour through many queries	Mitigated by rate limits and monitoring.
Membership inference	Learn whether a record was in the training data	A privacy risk for models trained on personal data.
Training data extraction	Make the model reproduce memorised text	Risk rises with duplicated data in training.

Adversarial examples	Inputs crafted to cause misclassification	Classic for vision models and classifiers.

6. The AI supply chain

Models, datasets, tokenizers, plugins and inference servers are all dependencies. Python pickle-based model files (.pt, .pkl, some .bin) can execute arbitrary code when loaded, so a downloaded model can be malware. Prefer the safetensors format, load models only from trusted registries or an internal mirror, pin versions and hashes, scan files, and generate a software bill of materials that includes models and datasets. Watch for typosquatted model names on public hubs.

7. MITRE ATLAS

MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) is a knowledge base of tactics and techniques used against AI systems, modelled on MITRE ATT&CK and illustrated with real case studies. It is useful for threat modelling and for mapping red-team findings to a shared vocabulary, for example reconnaissance of model capabilities, prompt injection, model evasion and exfiltration through AI inference APIs.